

Step 2 — Add a Long-Term Store, and a Node That Reads From It — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (the real `Store` API calls), **Advanced** (the namespace and prompt-injection details that make recall actually reliable). Read Basic first even if the `Store` idea already makes sense to you — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — Two memories, not one](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — Two memories, not one

Basic Version

The checkpointer from Step 1 and the Store you're adding now are two separate things that happen to sit in the same graph. The checkpointer is keyed by `thread_id` — one conversation. The Store is keyed by `user_id` — one person, no matter how many conversations they've ever had.

`recall_node` runs first, before the model ever sees the message, and its whole job is: look up what's known about this `user_id`, and hand it to the model as context.

Things to use: - `from langgraph.store.memory import InMemoryStore` . - A namespace tuple, like `(user_id, "memories")` — the same shape every time you read or write. -

`store.get(namespace, key)` for a fact you know the key of; `store.search(namespace, query=...)` when you want to find whatever's relevant without knowing the exact key. - A system message prepended to `state["messages"]`, built from whatever `recall_node` found.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The exact pieces:

- `store = InMemoryStore()` — build this once, at module load time in `store.py`, so the same instance is shared by every node that touches it in one run. (In-memory means it resets between runs of `python main.py` — that's fine for this step, since you're seeding a fact by hand each time to test recall. `PostgresStore` is the real production swap, same method calls, backed by a real database instead of a Python dict.)
- `store.put(namespace, key, value)` — `value` has to be a dict, not a bare string, e.g. `{"text": "Sam"}`, not `"Sam"`.
- `store.search(namespace, query="...")` returns a list of items, each with `.key` and `.value` — loop over it and pull `.value["text"]` (or however you shaped it) out of each one.
- A `LangGraph` node doesn't take extra arguments beyond `state` by default — to get the `store` into `recall_node`, either close over it (define `recall_node` inside a function that also has `store` in scope) or use `functools.partial(recall_node, store=store)` when calling `builder.add_node(...)`.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Think about exactly how `recall_node` gets its findings in front of the model. Prepending a `system` message to `state["messages"]` every single call means, over a long conversation, you'd end up with the same "known facts" message repeated many times if you're not careful — check whether a system message is already first in the list before adding another one, so `recall_node` running on every turn doesn't quietly duplicate its own injected context turn after turn.

Also think about what happens when the Store has nothing at all for a brand-new `user_id`. `store.search(...)` on an empty namespace returns an empty list — that's not an error, and `recall_node` shouldn't treat it as one. Don't inject a message like "I know nothing about this user" — that's not useful context, it's noise the model has to read past for no benefit. Only add a system message when there's actually something to say.

One more real thing worth testing: what if `recall_node` runs before the `user_id` even exists in `state` yet — the very first message of the very first thread for a brand-new user? Confirm your code handles a `user_id` with zero saved facts as cleanly as one with several, without a `KeyError` or a crash.

Things to try before Hint 2: - Seed a fact for `"sam"`, then run `recall_node` for a completely different `user_id` that has never been seen before — confirm it produces no injected message and doesn't crash. - Run the graph twice in a row on the same thread, and print `state["messages"]` after each call — confirm the injected "known facts" message doesn't appear twice.

Difference between Basic, Intermediate, and Advanced: Basic explains the two-memory-systems idea and names the pieces. Intermediate gives the real `InMemoryStore` calls and the closure/ `partial` trick for getting the store into a node function. Advanced is about the two failure modes that only show up with real use: a message injected more than once across multiple turns, and a brand-new user with nothing saved yet being handled gracefully instead of as a special error case.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
store.py:
    one shared InMemoryStore
    namespace_for(user_id) -> (user_id, "memories")

recall_node(state, store):
    look up facts for state["user_id"] in the store
    if any found, and no system message is first yet:
        put a system message first in messages, listing what was found
    return the (possibly updated) messages

graph: START -> recall_node -> chat_node -> END

main.py:
    fixed user_id, NEW thread_id every run
    seed one fact by hand before the loop, for testing
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Here's almost the whole thing — type it out yourself and adjust to your own field names:

```
# store.py
from langgraph.store.memory import InMemoryStore

_store = InMemoryStore()

def get_store():
    return _store

def namespace_for(user_id: str) -> tuple:
    return (user_id, "memories")
```

```
# state.py (adds to Step 1's ChatState)
from typing import Annotated, TypedDict
from langgraph.graph.message import add_messages

class ChatState(TypedDict):
    messages: Annotated[list, add_messages]
    user_id: str
```

Your turn: write `recall_node(state, store)` in `graph.py`, using `store.search(namespace_for(state["user_id"]), query="facts about this user")`, and wire it in before `chat_node` using `functools.partial`.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Fill in the duplicate-check and empty-result handling yourself, using this skeleton:

```
# graph.py (relevant part only)
from functools import partial
from langgraph.graph import StateGraph, START, END
from state import ChatState
from store import namespace_for

def recall_node(state: ChatState, store) -> dict:
    namespace = namespace_for(state["user_id"])
    found = store.search(namespace, query="facts about this user")

    if not found:
        return {} # nothing known yet -- add nothing, don't crash, don't inject

    already_has_recall = (
        state["messages"]
        and getattr(state["messages"][0], "type", None) == "system"
    )
    if already_has_recall:
        return {} # your turn: think about whether this is really the right check
                # what if new facts were saved since the last time this ran?

    facts_text = ", ".join(item.value.get("text", "") for item in found)
    system_message = {"role": "system", "content": f"Known facts about this user: {facts_text}"}
    return {"messages": [system_message]}

def build_graph(checkpointer, store):
    builder = StateGraph(ChatState)
    builder.add_node("recall_node", partial(recall_node, store=store))
    builder.add_node("chat_node", chat_node)
    builder.add_edge(START, "recall_node")
    builder.add_edge("recall_node", "chat_node")
    builder.add_edge("chat_node", END)
    return builder.compile(checkpointer=checkpointer)
```

The comment marks a real open question on purpose: checking "is the first message already a system message" is a simple guard against duplicating within one thread, but it also means a fact

saved mid-conversation won't get added until a new thread starts. Decide, and write down, whether that trade-off is acceptable for this project (it is, for a teaching project) or whether you'd want `recall_node` to re-check every turn in a production system.

Compare your finished `store.py`, `state.py`, and `recall_node` against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic sketches the flow in plain words. Intermediate gives the real `InMemoryStore` setup and the state changes. Advanced gives the actual `recall_node` code, including the empty-result guard and the once-per-thread duplicate guard, and asks you to think honestly about what that guard does and doesn't cover.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)